OUTPUT ANALYSIS

Threaded BST Predecessor and Successor • Double-Threaded Binary Search Tree in C++

1. Time Complexity Analysis

Let **h** be the height of the tree and **n** the number of nodes.

Operation	Time Complexity	Reason
Insert Node	**O(h)**	Moves down from the root to find the position; linking the node is O(1).
Search Node	**O(h)**	Moves down one level per comparison.
Find Successor	**O(h)**	Right thread gives it in O(1); otherwise go to the leftmost node of the right subtree.
Find Predecessor	**O(h)**	Left thread gives it in O(1); otherwise go to the rightmost node of the left subtree.
Inorder Traversal	**O(n)**	Each node is visited once using threads, without a stack.

2. Space Complexity Analysis

Threads reuse the NULL pointers, so no extra pointers are needed.

Total Space	**O(n)**	The tree stores n nodes.
Extra Space per Operation	**O(1)**	Only a few pointers are used; no recursion or stack.

Interpretation: Finding a node, its predecessor or its successor takes O(h) time (O(log n) for a balanced tree). Threads make the inorder traversal run in O(n) time with O(1) extra space.